

Streams 2

Übungen

Stream – Aufgabe Odd

Löse die folgende Aufgabe mithilfe von Streams.

Nutze folgende Liste:

```
List.of(11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1);
```

1. Sammle alle ungeraden Zahlen in einer neuen Collection.
2. Gebe die neue Collection in aufsteigender Reihenfolge in der Konsole aus.

Ergebnis:

```
[1, 3, 5, 7, 9, 11]
```

Stream – Lösung Odd

```
public class Solution {  
    public static void main(String[] args) {  
        var numbers = List.of(11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1);  
  
        List<Integer> oddNumbersDescending = numbers.stream()  
            .filter(n -> n % 2 == 1)  
            .sorted()  
            .toList();  
  
        System.out.println(oddNumbersDescending);  
    }  
}
```

Stream – Aufgabe Duplikate

Löse die folgende Aufgabe mithilfe von Streams.

Nutze folgende Liste:

```
List.of(1, 3, 8, 3, 9, 7, 1, 5, 8, 3, 9, 4, 7, 4, 6, 0, 8, 3, 7);
```

1. Prüfe zunächst, ob die Liste Duplikate enthält.
2. Falls keine Duplikate existieren, gebe "Die Liste enthält keine Duplikate." in der Konsole aus.
3. Falls Duplikate existieren, verwende ein Stream, um eine neue Liste zu erzeugen.
Die neue Liste enthält alle Elemente ohne Duplikate. Gebe die neue Liste in der Konsole aus.

Ergebnis:

```
[1, 3, 8, 9, 7, 5, 4, 6, 0]
```

Stream – Lösung Duplikate

```
public class Solution {  
    public static void main(String[] args) {  
        List<Integer> numbers = List.of(1, 3, 8, 3, 9, 7, 1, 5, 8, 3, 9, 4, 7, 4, 6, 0, 8, 3, 7);  
  
        var distinctNumbers = numbers.stream().distinct().toList();  
        boolean containsDuplicates = distinctNumbers.size() != numbers.size();  
        if (containsDuplicates) {  
            System.out.println(distinctNumbers);  
        } else {  
            System.out.println("Die Liste enthält keine Duplikate.");  
        }  
    }  
}
```

Stream – Aufgabe Städte

Gegeben ist folgender Code:

```
var persons = Stream.of(  
    new Person("Elon", new Address("Mönckebergstraße", "Hamburg")),  
    new Person("Olaf", new Address("Rathausstraße", "Hamburg")),  
    new Person("Angela", new Address("Schillerstraße", "Berlin")),  
    new Person("Markus", new Address("Hofbräuhaus", "München")),  
    new Person("Annalena", new Address("Hauptstraße", "Berlin")));
```

1. Erstelle die fehlenden Klassen / Records.
2. Nutze den Stream, um eine Liste zu erzeugen, die alle vorhandenen Städte ohne Duplikate enthält.
3. Gebe diese Liste in der Konsole aus.

Ergebnis:

```
[Hamburg, Berlin, München]
```

Stream – Lösung Städte

```
public class Cities {  
    public static void main(String[] args) {  
        var persons = Stream.of(  
            new Person("Elon", new Address("Mönckebergstraße", "Hamburg")),  
            new Person("Olaf", new Address("Rathausstraße", "Hamburg")),  
            new Person("Angela", new Address("Schillerstraße", "Berlin")),  
            new Person("Markus", new Address("Hofbräuhaus", "München")),  
            new Person("Annalena", new Address("Hauptstraße", "Berlin")));  
  
        var cities = persons.map(person ->  
                               person.address().city()).distinct().toList();  
        System.out.println(cities);  
    }  
}
```

Stream – Aufgabe Supermarkt

Gegeben ist folgender Code:

```
var sortiment = List.of(  
    new Artikel("Banane", 100, 1.39),  
    new Artikel("Tomate", 200, 0.29),  
    new Artikel("Gurke", 50, 0.99),  
    new Artikel("Apfel", 300, 0.59),  
    new Artikel("Aubergine", 10, 2.19));
```

```
record Artikel(  
    String name,  
    int aufLager,  
    double preis) {  
}
```

1. Erstelle die fehlenden Klassen / Records.
2. Es soll eine Übersicht (Report) für das Management erstellt werden. Nutze Streams, um die verschiedenen Ausgaben zu erzeugen und auf der Konsole auszugeben (siehe Ergebnis).

Ergebnis:

```
Anzahl Artikel auf Lager insgesamt: 660  
Lagerwert insgesamt: 445,40 Euro  
Lagerwert pro Artikel:  
Banane: 139,00 Euro  
Tomate: 58,00 Euro  
Gurke: 49,50 Euro  
Apfel: 177,00 Euro  
Aubergine: 21,90 Euro
```


Stream – Lösung Supermarkt

// Für diese Aufgabe gibt es viele denkbare Lösungsansätze

```
public class Supermarkt {  
    public static void main(String[] args) {  
        var sortiment = List.of(  
            new Artikel("Banane", 100, 1.39),  
            new Artikel("Tomate", 200, 0.29),  
            new Artikel("Gurke", 50, 0.99),  
            new Artikel("Apfel", 300, 0.59),  
            new Artikel("Aubergine", 10, 2.19));  
  
        var artikelAufLager = sortiment.stream().mapToInt(artikel -> artikel.aufLager()).sum();  
        System.out.printf("Anzahl Artikel auf Lager insgesamt: %d\n", artikelAufLager);  
  
        var lagerwert = sortiment.stream().mapToDouble(artikel -> artikel.aufLager() * artikel.preis()).sum();  
        System.out.printf("Lagerwert insgesamt: %.2f Euro\n", lagerwert);  
  
        System.out.println("Lagerwert pro Artikel: ");  
        sortiment.stream()  
            .map(artikel -> String.format(  
                "%s: %.2f Euro",  
                artikel.name(), artikel.aufLager() * artikel.preis()))  
            .forEach(artikelMitLagerwert -> System.out.println(artikelMitLagerwert));  
    }  
}
```

Stream – Bonusaufgabe Supermarkt

Erweitere deine Lösung aus der Aufgabe Supermarkt.

Gebe die Artikel aufsteigend sortiert nach ihrem Lagerwert aus.

Ergebnis:

```
Anzahl Artikel auf Lager insgesamt: 660  
Lagerwert insgesamt: 445,40 Euro  
Lagerwert pro Artikel:  
Aubergine: 21,90 Euro  
Gurke: 49,50 Euro  
Tomate: 58,00 Euro  
Banane: 139,00 Euro  
Apfel: 177,00 Euro
```

Stream – Bonusaufgabe Supermarkt

```
// Viele Lösungsvarianten sind hier möglich
public class Supermarkt {
    public static void main(String[] args) {
        var sortiment = List.of(
            new Artikel("Banane", 100, 1.39),
            new Artikel("Tomate", 200, 0.29),
            new Artikel("Gurke", 50, 0.99),
            new Artikel("Apfel", 300, 0.59),
            new Artikel("Aubergine", 10, 2.19));

        var artikelAufLager = sortiment.stream().mapToInt(Artikel::aufLager).sum();
        System.out.printf("Anzahl Artikel auf Lager insgesamt: %d\n", artikelAufLager);

        var lagerwert = sortiment.stream().mapToDouble(artikel -> artikel.aufLager() * artikel.preis()).sum();
        System.out.printf("Lagerwert insgesamt: %.2f Euro\n", lagerwert);

        System.out.println("Lagerwert pro Artikel: ");
        sortiment.stream()
            // .sorted(Comparator.comparingDouble(artikel -> artikel.aufLager() * artikel.preis()))
            .sorted((artikel1, artikel2) -> Double.compare(artikel1.aufLager() * artikel1.preis(),
                                                         artikel2.aufLager() * artikel2.preis()))
            .map(artikel -> String.format(
                "%s: %.2f Euro",
                artikel.name(), artikel.aufLager() * artikel.preis()))
            .forEach(System.out::println);
    }
}
```

Stream – Aufgabe Match

Löse die folgende Aufgabe mithilfe von Streams.

Nutze folgende Liste:

```
List.of("Anna", "Berta", "Clara", "Diana", "Eva")
```

1. Prüfe, ob mindestens ein Name existiert, bei dem der vierte Buchstabe ein „a“ ist. Gebe das Ergebnis in der Konsole aus.
2. Prüfe, ob alle Namen mit dem Buchstaben „a“ enden. Gebe das Ergebnis in der Konsole aus.

Ergebnis:

```
true  
true
```

Stream – Lösung Match

```
public class Solution {  
    public static void main(String[] args) {  
        List<String> names = List.of("Anna", "Berta", "Clara", "Diana", "Eva");  
  
        boolean anyFourthLetterIsA = names.stream()  
            .anyMatch(n -> n.length() > 3 && n.charAt(3) == 'a');  
        System.out.println(anyFourthLetterIsA);  
  
        boolean allEndWitha = names.stream()  
            .allMatch(n -> n.endsWith("a"));  
        System.out.println(allEndWitha);  
    }  
}
```

Stream – Aufgabe Moin

Löse die folgende Aufgabe mithilfe von Streams.

Erstelle folgende Liste:

```
List.of("m", "o", "i", "n");
```

1. Wandle die Buchstaben in Großbuchstaben um.
2. Füge die einzelnen Buchstaben zu einem String zusammen.
3. Gebe den String in der Konsole aus.

Ergebnis:

MOIN

Stream – Lösung Moin

```
public class Solution {  
    public static void main(String[] args) {  
        var letters = List.of("m", "o", "i", "n");  
  
        String uppercaseString = letters.stream()  
            .map(String::toUpperCase)  
            // .map(letter -> letter.toUpperCase())  
            .reduce("", (substring, letter) -> substring + letter);  
  
        System.out.println(uppercaseString);  
    }  
}
```

Stream – Aufgabe Mean Numbers

Löse die folgende Aufgabe mithilfe von Streams.

Erstelle folgende Liste:

```
List.of(9, 1, 5, 3, 2, 4, 6);
```

1. Berechne zunächst die Summe aller Zahlen.
2. Teile dann jede einzelne Zahl durch die Summe.
3. Gebe die neuen Zahlen in der Konsole aus.

Ergebnis:

```
[0.3, 0.03333333333333333, 0.16666666666666666, 0.1, 0.06666666666666667,  
0.13333333333333333, 0.2]
```


Stream – Lösung Mean Numbers

```
public class Solution {  
    public static void main(String[] args) {  
        List<Integer> numbers = List.of(9, 1, 5, 3, 2, 4, 6);  
  
        double sum = numbers.stream().reduce(0, (subtotal, number) -> subtotal + number);  
        //double sum = numbers.stream().mapToInt(Integer::intValue).sum();  
        var fractions = numbers.stream()  
            .map(n -> (double) n / sum)  
            .toList();  
  
        System.out.println(fractions);  
    }  
}
```